INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	Construcción de software				
TÍTULO DE LA PRÁCTICA:	*UML como código*				
NÚMERO DE PRÁCTICA:	*05*	**AÑO LECTIVO:**	*2026B*	**NRO. SEMESTRE:**	*VI*
FECHA DE PRESENTACIÓN	*10/10/26*	**HORA DE PRESENTACIÓN**			
INTEGRANTE (s): Oliva Valdivia David Alexander				**NOTA:**	
DOCENTE(s): *Mg. Antonio Arroyo Paz*					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

Repositorio: `github.com/DavidOliva06/cs-2026b-lab04-grupo06`. La entrega original es la etiqueta `v0.5-diseno`; esta versión del informe corresponde a la etiqueta `v0.5.1-diseno`, que agrega el commit de E8 (no incluido en `v0.5-diseno`) y las correcciones de la revisión. Todas las rutas son relativas a la raíz del repositorio.

Tabla 1. Commits que respaldan cada ejercicio.

Ejercicio	Commit	Mensaje	Fecha
E1	`130a1b5`	E1: diagrama de clases.	2026-10-11
E2	`c4ba190`	E2: secuencia.	2026-10-11
E3	`607c3cc`	E3: Estados	2026-10-11
E4	`b69f3e9`	E4: Diagrama de actividades	2026-10-11
E5	`1968714`	E5: Paquetes	2026-10-11
E6	`23ae80e`	E6: Round-Trip con IA	2026-10-11
E7	`6ecb88c`	E7: Bitacora de usa de IA (etiqueta `v0.5-diseno`)	2026-10-11
E8	`556f825`	E8: README.txt	2026-10-11
Revisión	`b84689c`	Lab 05: correcciones tras la revisión del informe (etiqueta `v0.5.1-diseno`)	2026-11-13

E1: Historia de usuario y diagrama de clases

1. **En historia.md, escriba la historia de usuario de la columna E2 de su caso, con 2 o más criterios de aceptación en formato Dado / Cuando / Entonces.**

Archivo `docs/design/historia.md` (cuatro criterios de aceptación):

```
## HU-01: Enviar un reporte sin señal y sincronizarlo
Como ciudadano afectado por un sismo, quiero registrar un reporte de daños con foto
y ubicación aunque no tenga señal, para que Defensa Civil lo reciba en cuanto mi
celular recupere la conexión.

1. Dado que el ciudadano aceptó el consentimiento y su celular no tiene señal,
   cuando completa el reporte con foto y ubicación y pulsa "Enviar", entonces la app
   guarda el reporte en el celular y lo muestra como "Pendiente de envío".
2. Dado que existe al menos un reporte "Pendiente de envío", cuando el celular
   recupera la señal, entonces la app lo envía sola y el estado cambia a "Enviado"
   en un máximo de 15 minutos.
3. Dado que un reporte llegó al servidor pero se perdió la confirmación, cuando la
   app lo reenvía, entonces el servidor conserva un solo reporte.
4. Dado que el ciudadano no aceptó el consentimiento, cuando intenta crear un
   reporte, entonces la app no captura ubicación ni foto y le muestra el consentimiento.
```

2. **Adapte el Prompt IA 1 con su historia y su ADR-001, y genere un primer diagrama de clases. Registre el prompt en bitacora-ia.md.**

Prompt usado (registrado como interacción 1 en `docs/design/bitacora-ia.md`):

```
Actúa como diseñador de software orientado a objetos.
Contexto: módulo almacenamiento de un monolito modular con ingesta asíncrona
como estilo arquitectónico (ADR-001).
Historia y criterios: el archivo historia.md.
Tarea: genera un diagrama de clases en PlantUML con atributos tipados, operaciones,
multiplicidades, una enumeración para el estado del pedido y una interfaz (puerto)
para la pasarela de pagos.
Formato: solo el código PlantUML. No agregues clases que no se deriven de la
historia; si asumes algo, indícalo en un comentario.
```

El prompt contenía un error ("pedido" y "pasarela de pagos" no existen en el caso). La IA lo advirtió y usó los equivalentes del dominio: `EstadoReporte` y el puerto `PuertoEnvioReportes`.

3. **Revise la propuesta: elimine clases que no se deriven de la historia, corrija relaciones (composición o asociación, herencia o realización) y agregue las multiplicidades.**

No hubo clases que eliminar: cada clase de la propuesta sale de la historia (ciudadano, consentimiento, reporte, foto, ubicación, envío y recepción). Sí se corrigieron ocho problemas:

Tabla 2. Revisión de la propuesta de la IA.

Propuesta de la IA	Decisión	Justificación
`AlmacenReportes` compartido por celular y servidor	Separar en `AlmacenLocalReportes` y `AlmacenServidorReportes`.	`buscarPorEstado` solo sirve en el celular y `existe` solo en el servidor; cada lado dependía de operaciones que no usa.
`existe(id)` seguido de `guardar(reporte)`	Una sola operación atómica: `guardarSiNoExiste(reporte)`.	Dos reintentos simultáneos podían pasar ambos la comprobación y duplicar el reporte (criterio 3).
Atributo `estado` y asociación con `EstadoReporte` a la vez	Dejar solo la asociación `"0..*" --> "1"`.	Expresaban lo mismo dos veces.
`actualizarEstado(id, estado)` en el puerto	Eliminarla.	Competía con la operación de `Reporte` y no registraba la fecha.

`aceptado` y `fechaAceptacion` en `Consentimiento`	Dejar solo `fechaAceptacion [0..1]`.	Eran redundantes y podían contradecirse.
`fechaEnvio` obligatoria	Multiplicidad `[0..1]`.	No existe mientras el reporte está pendiente.
Asociaciones sin multiplicidad	Composición `"1" *-- "1"` con `Foto`, `Ubicacion` y `Consentimiento`; asociación `"1" -- "0..*"` con `Reporte`.	La foto y la ubicación no existen sin su reporte; un ciudadano registra muchos reportes.
`recibir()` y `enviar()` devuelven `Boolean` sin explicar	Documentar que `true` significa "el servidor tiene el reporte".	Un reporte repetido también debe marcarse como recibido (criterio 3).

4. El diagrama final debe tener: 6 o más clases, 1 interfaz (puerto hacia un servicio externo), 1 enumeración, atributos con tipo y visibilidad, y operaciones.

Tabla 3. Verificación de los requisitos del diagrama de clases (versión v0.5.1-diseno).

Requisito	¿Cumple?	Evidencia en docs/design/clases.puml
6 o más clases	Sí (12)	`Ciudadano`, `Consentimiento`, `Reporte`, `Foto`, `Ubicacion`, `ServicioReportes`, `ServicioIngesta`, `FormularioReporte`, `ControladorReportes` y tres adaptadores.
1 interfaz hacia un servicio externo	Sí (3)	`PuertoEnvioReportes <<puerto>>` (hacia el servidor), `AlmacenLocalReportes` y `AlmacenServidorReportes`.
1 enumeración	Sí	`EstadoReporte` con seis valores: BORRADOR, ENCOLADO, RECIBIDO, VERIFICADO, ATENDIDO, RECHAZADO.
Atributos con tipo y visibilidad	Sí	Por ejemplo `- id: UUID`, `- fechaRecepcion: DateTime [0..1]`, `- latitud: Double`.
Operaciones	Sí	Por ejemplo `+ encolar(): void`, `+ marcarRecibido(fecha: DateTime): void`, `+ guardarSiNoExiste(reporte: Reporte): Boolean`.

5. Renderice con PlantUML, guarde el PNG en img/ y haga commit con el mensaje E1: diagrama de clases.

PNG en `docs/design/img/DiagramadeClases.png`; commit `130a1b5` "E1: diagrama de clases.". La Figura 1 es la versión final, que además incluye las clases agregadas por la regla C1 (E2) y los adaptadores del round-trip (E6).

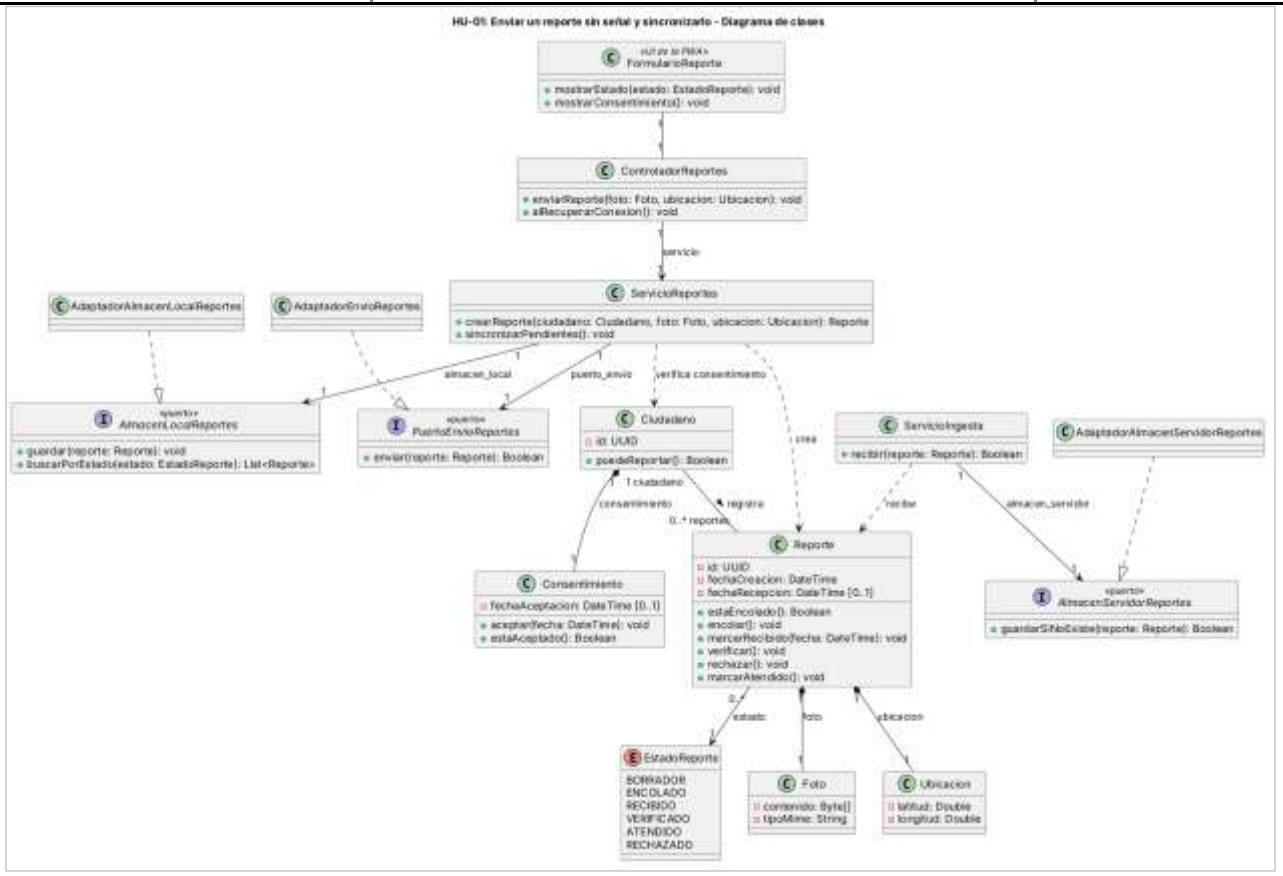


Figura 1. Diagrama de clases de la HU-01 (docs/design/clases.puml).

E2: Diagrama de secuencia del flujo crítico

1. Elija el escenario principal de su historia (por ejemplo, "reservar cita con cupo disponible").

Escenario: **registrar un reporte sin señal y sincronizarlo al recuperar la conexión**. Cubre los cuatro criterios de aceptación de la HU-01.

2. Dibuje en PlantUML 5 o más líneas de vida: actor, interfaz (UI o API), controlador, servicio, entidades y puertos.

Tabla 4. Líneas de vida de docs/design/secuencia-reporte.puml (10 en total).

Tipo pedido	Líneas de vida
Actor	`Ciudadano`
Interfaz (UI)	`FormularioReporte` (boundary)
Controlador	`ControladorReportes` (control)
Servicios	`ServicioReportes` (celular) y `ServicioIngesta` (servidor)
Entidades	`ciudadano: Ciudadano` y `reporte: Reporte`
Puertos	`AlmacenLocalReportes`, `PuertoEnvioReportes` y `AlmacenServidorReportes`

3. Incluya un fragmento `alt` para el caso de éxito y el de error, y un fragmento `loop` u `opt`.

- `alt` (líneas 26 a 40): "consentimiento aceptado" (éxito, criterio 1) frente a "consentimiento no aceptado" (error, criterio 4).
- `loop` (líneas 51 a 65): "por cada reporte pendiente" se envía y se guarda sin duplicar (criterios 2 y 3).

- opt (líneas 59 a 64): solo si el servidor confirmó, el reporte se marca como recibido.

4. Incluya al menos un mensaje asíncrono (->>), por ejemplo una notificación, y los mensajes de retorno.

- Asíncronos: `UI ->> CTRL: alRecuperarConexion()` (línea 46) y `CTRL ->> UI: mostrarEstado(RECIBIDO)` (línea 68).
- Retornos (`-->`): 16 mensajes, por ejemplo `SERV --> ING: Boolean (false si ya existía)` y `ENVIO --> SVC: confirmado`.

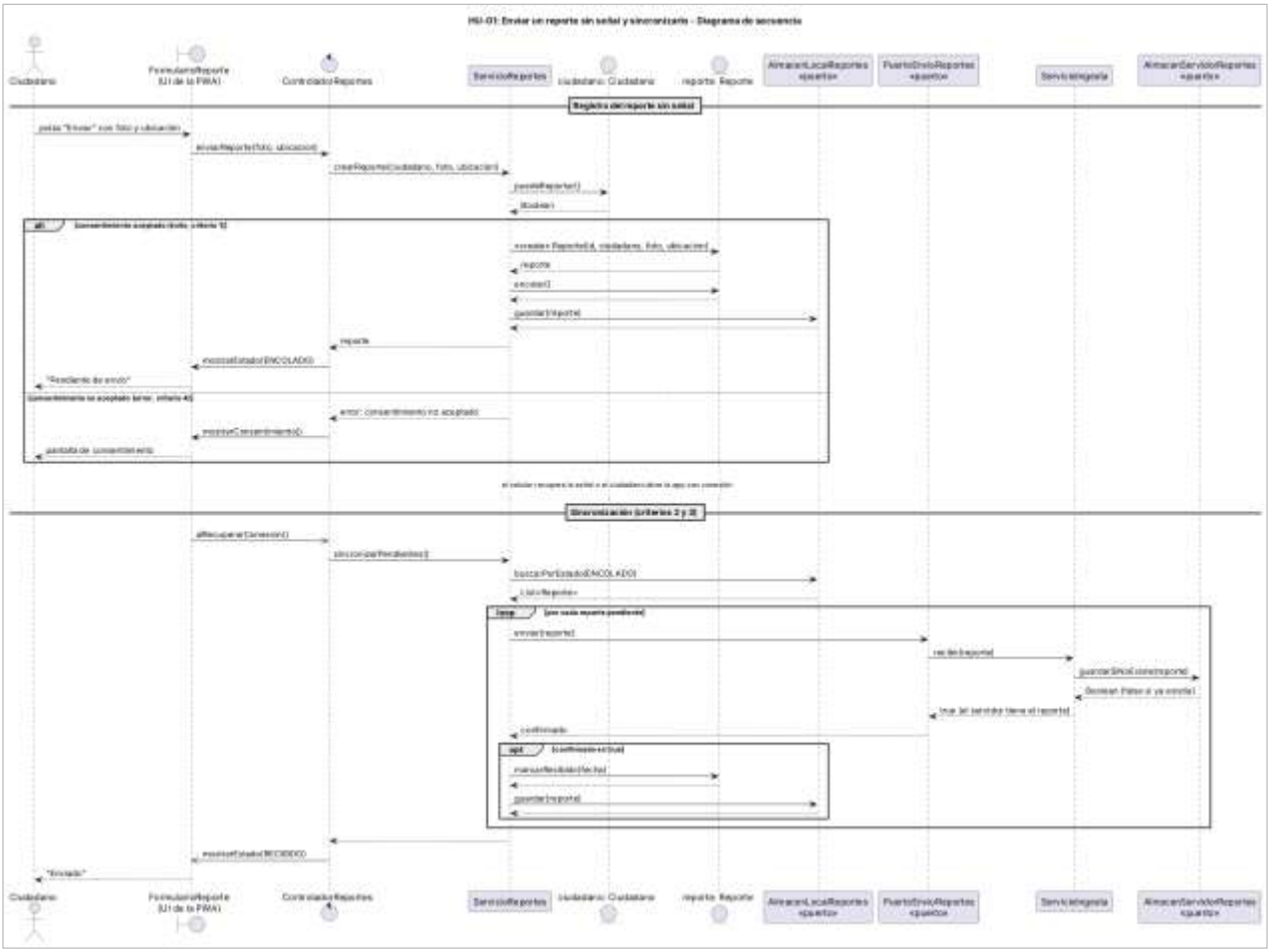


Figura 2. Diagrama de secuencia corregido (docs/design/secuencia-reporte.puml).

5. Verifique la regla C1: si un mensaje no existe como operación, agréguela al diagrama de clases. Haga commit con el mensaje E2: secuencia.

La verificación se hizo con el script `docs/design/verificar_c1.py`, que lee cada mensaje de llamada y busca la operación en la clase receptora de `clases.puml`. En la versión entregada (`v0.5-diseno`) encontró **6 mensajes sin operación**: `enviarReporte()` y `alRecuperarConexion()` (no existía `ControladorReportes`), `mostrarEstado()` y `mostrarConsentimiento()` (no existía `FormularioReporte`) y `marcarEnviado(fecha)`, que en E3 pasó a llamarse `marcarRecibido(fecha)` sin actualizar la secuencia. Tras la corrección (`b84689c`) el resultado es 0:

```
$ python docs/design/verificar_c1.py docs/design/secuencia-reporte.puml docs/design/clases.puml
```

Tabla 5. Salida de verificar_c1.py sobre la versión corregida.

Línea	Mensaje	Clase receptora	¿Existe la operación?
21	`enviarReporte(foto, ubicacion)`	ControladorReportes	Sí

22	`crearReporte(ciudadano, foto, ubicacion)`	ServicioReportes	Sí
23	`puedeReportar()`	Ciudadano	Sí
29	`encolar()`	Reporte	Sí
31	`guardar(reporte)`	AlmacenLocalReportes	Sí
34	`mostrarEstado(ENCOLADO)`	FormularioReporte	Sí
38	`mostrarConsentimiento()`	FormularioReporte	Sí
46	`alRecuperarConexion()`	ControladorReportes	Sí
47	`sincronizarPendientes()`	ServicioReportes	Sí
48	`buscarPorEstado(ENCOLADO)`	AlmacenLocalReportes	Sí
52	`enviar(reporte)`	PuertoEnvioReportes	Sí
53	`recibir(reporte)`	ServicioIngesta	Sí
54	`guardarSiNoExiste(reporte)`	AlmacenServidorReportes	Sí
60	`marcarRecibido(fecha)`	Reporte	Sí
62	`guardar(reporte)`	AlmacenLocalReportes	Sí
68	`mostrarEstado(RECIBIDO)`	FormularioReporte	Sí

Commit original: `c4ba190` "E2: secuencia.". Corrección: `b84689c`.

E3: Máquina de estados de la entidad principal

1. Tome la entidad sugerida para su caso y escriba el diagrama en Mermaid (stateDiagram-v2). Valídelo en mermaid.live.

Entidad: `Reporte`. Archivo `docs/design/estado-estadoReporte.mmd`; el código se renderiza sin errores (Figura 3) y está embebido en el README.

```
stateDiagram-v2
    [*] --> Borrador : crearReporte() [consentimiento aceptado]
    Borrador --> Encolado : encolar() [foto y ubicación completas]
    Encolado --> Encolado : enviar() [servidor no confirma]
    Encolado --> Recibido : recibir() [servidor confirma el reporte]
    Recibido --> Verificado : verificar() [brigadista confirma el daño]
    Recibido --> Rechazado : rechazar() [reporte falso o repetido]
    Verificado --> Atendido : marcarAtendido() [brigada terminó la atención]
    Atendido --> [*]
    Rechazado --> [*]
```

2. Incluya 5 o más estados, el estado inicial [*], al menos un estado final y 2 o más guardas entre corchetes.

Tabla 6. Verificación de los requisitos del diagrama de estados.

Requisito	¿Cumple?	Evidencia
5 o más estados	Sí (6)	Borrador, Encolado, Recibido, Verificado, Atendido y Rechazado; coinciden con `EstadoReporte`.
Estado inicial `[*]`	Sí	`[*] --> Borrador`
Al menos un estado final	Sí (2)	`Atendido --> [*]` y `Rechazado --> [*]`
2 o más guardas	Sí (7)	Una por transición, por ejemplo `[foto y ubicación completas]`.

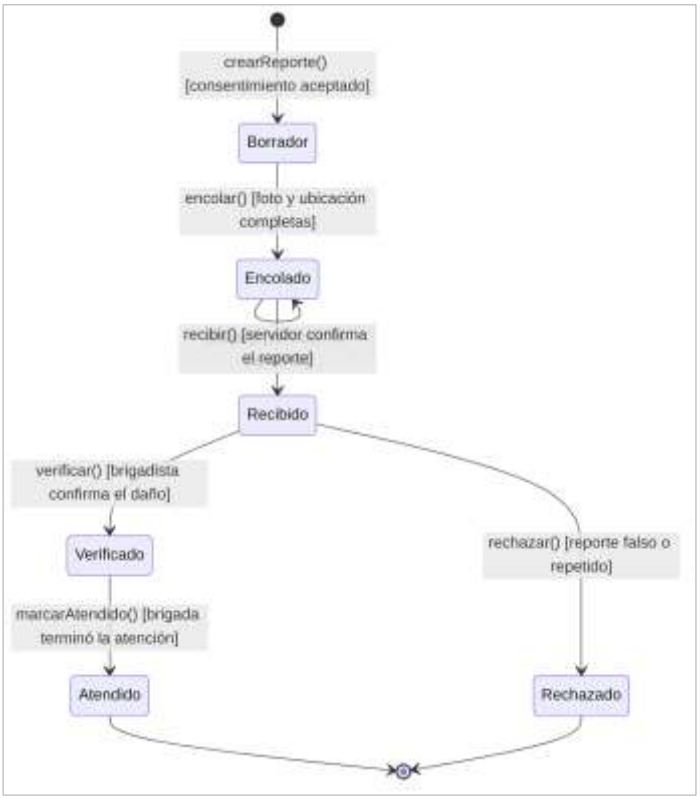


Figura 3. Máquina de estados de Reporte (docs/design/estado-estadoReporte.mmd).

3. Nombre cada transición con la operación de la clase que la provoca (C2). Haga commit con el mensaje E3: estados.

Tabla 7. Cada transición y la operación que la provoca en clases.puml.

Transición	Operación	Clase que la provoca	¿Existe en clases.puml?
[*] → Borrador	`crearReporte()`	ServicioReportes	Sí
Borrador → Encolado	`encolar()`	Reporte	Sí
Encolado → Encolado	`enviar()`	PuertoEnvioReportes	Sí
Encolado → Recibido	`recibir()`	ServicioIngesta	Sí
Recibido → Verificado	`verificar()`	Reporte	Sí
Recibido → Rechazado	`rechazar()`	Reporte	Sí
Verificado → Atendido	`marcarAtendido()`	Reporte	Sí

Para cumplir C2, al escribir este diagrama se pasó `EstadoReporte` de dos a seis valores y se agregaron `verificar()`, `rechazar()` y `marcarAtendido()` a `Reporte`. La revisión de consistencia (E7) dejó una observación abierta: `enviar()` y `recibir()` son operaciones de otras clases y no de `Reporte`. Commit: `607c3cc` "E3: Estados".

E4: Diagrama de actividades

1. Modele el proceso de la última columna de la tabla de casos con PlantUML (|Partición|).

Proceso: **priorización y asignación de brigadas**, en `docs/design/actividades-brigada.puml` (Figura 4). El ciclo `repeat ... repeat while (¿Continúa la emergencia?)` hace que el proceso se repita mientras dure la emergencia.

2. Incluya 3 o más particiones, 2 o más decisiones (if / else) y una ejecución paralela (fork / end fork).

Tabla 8. Verificación de los requisitos del diagrama de actividades.

Requisito	¿Cumple?	Evidencia en actividades-brigada.puml
3 o más particiones	Sí (3)	`|Sistema|`, `|Coordinador|` y `|Brigadista|`
2 o más decisiones	Sí (5)	¿Hay zonas con reportes sin atender?, ¿Hay una brigada disponible?, ¿Quedan reportes por revisar?, ¿El daño es real? y ¿Continúa la emergencia?
Ejecución paralela	Sí	`fork`: actualizar el mapa de zonas ‖ publicar la asignación en la vista del brigadista.

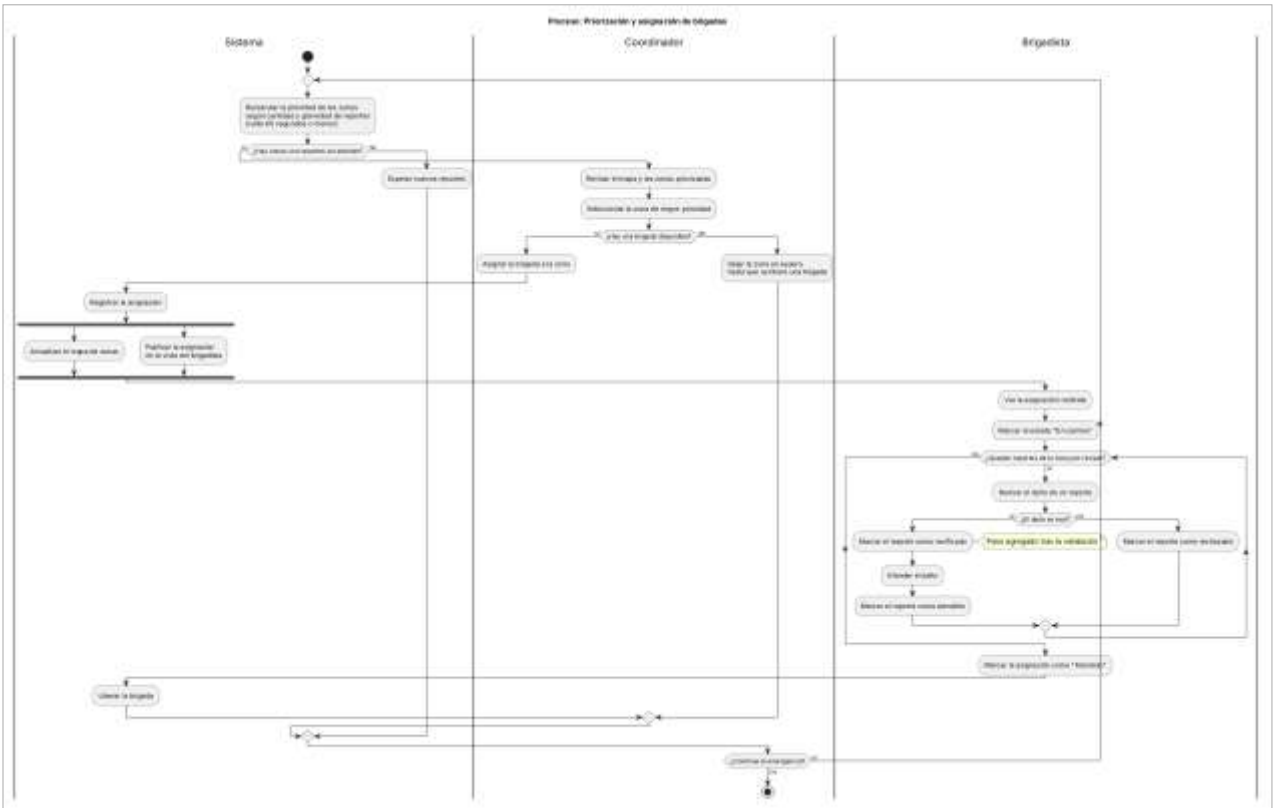


Figura 4. Proceso de priorización y asignación de brigadas (versión corregida).

3. Valide el flujo con alguien que conozca el proceso real y anote un cambio que surgió de esa revisión.

Validación con una persona. El flujo se revisó con un compañero del curso que conoce el caso. Su observación fue que un reporte no puede pasar directamente a "Atendido": primero el brigadista debe confirmar que el daño es real. **Cambio:** dentro del ciclo por reporte se agregó la actividad "Marcar el reporte como verificado" antes de atenderlo, con lo que el proceso respeta la secuencia Recibido → Verificado → Atendido del diagrama de estados (nota "Paso agregado tras la validación" en la Figura 4). **Limitación:** el revisor no trabaja en Defensa Civil; la validación con personal del proceso real queda pendiente.

Revisión complementaria con IA. Antes de esa validación, una revisión asistida por IA encontró otros problemas en la primera versión:

Tabla 9. Cambios entre la primera versión y la versión validada.

Problema de la primera versión	Cambio aplicado
El flujo terminaba tras atender una zona; la zona en espera nunca se retomaba.	Ciclo `repeat` mientras continúe la emergencia.

La decisión "¿los daños son reales?" se aplicaba a toda la zona en bloque.	Ciclo `while` que revisa reporte por reporte.
Faltaba el paso por "Verificado" (observación del compañero).	Actividad "Marcar el reporte como verificado".
El paralelismo era artificial: registrar, actualizar y publicar iban a la vez.	Primero se registra la asignación; en paralelo solo quedan el mapa y la publicación.
No se indicaba qué dispara el recálculo (QA-02 exige 60 s o menos).	"Recalcular la prioridad... (cada 60 segundos o menos)".
Sin salida si la brigada no responde.	Pendiente: reasignación por tiempo de espera.

Commit: `b69f3e9` "E4: Diagrama de actividades"; la nota de la validación se agregó en `b84689c`.

E5: Diagrama de paquetes

1. Cree un paquete por cada módulo del ADR-001, más un paquete compartido si lo necesita.

Tabla 10. Paquetes de docs/design/paquetes.puml.

Paquete	Origen	Contenido
reportes	Módulo del ADR-001	`Reporte`, `EstadoReporte`, `ServicioIngesta`, `AlmacenServidorReportes`, `ConsultaReportes`
mapa	Módulo del ADR-001	`ServicioMapa`
priorización	Módulo del ADR-001	`ServicioPriorizacion`, `ZonaPriorizada`, `ConsultaZonasAsignadas <<puerto>>`
brigadas	Módulo del ADR-001	`Brigada`, `Asignacion`, `ServicioAsignacion`
compartido	Paquete compartido	`Ubicacion`, `Zona`, `ColaTrabajos <<puerto>>`

2. Dibuje y etiquete las dependencias (qué clase o puerto se usa) y agregue una nota con la regla de dependencias.

Las nueve dependencias están etiquetadas con lo que se usa (por ejemplo `mapa ..> priorizacion : usa ZonaPriorizada`, `reportes ..> compartido : usa Ubicacion y ColaTrabajos`). La nota del diagrama dice:

```
Regla de dependencias
1. Las dependencias van en un solo sentido:
   mapa y brigadas -> priorización -> reportes -> compartido.
2. El paquete compartido no depende de ningún módulo.
3. Un módulo usa de otro solo sus puertos y clases públicas,
   nunca sus tablas ni sus clases internas.
4. No se permiten ciclos. Si un módulo necesita datos de otro
   que ya depende de él, define un puerto y el otro lo implementa.
```

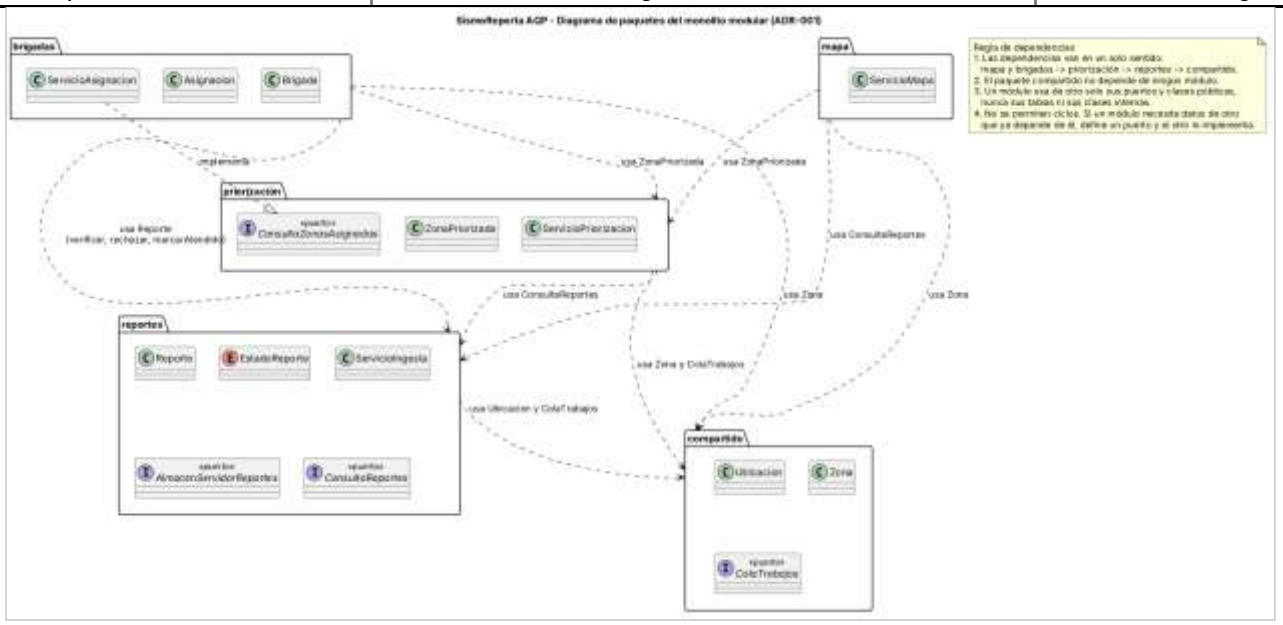


Figura 5. Diagrama de paquetes del monolito modular (docs/design/paquetes.puml).

3. Verifique la regla C4 (sin ciclos). Si encuentra un ciclo, resuélvalo con un puerto (interfaz) y registre la decisión en un nuevo ADR.

Ciclo encontrado: brigadas necesita `ZonaPriorizada` de priorización, y priorización necesitaba saber qué zonas ya tienen brigada. **Resolución:** priorización define el puerto `ConsultaZonasAsignadas` y brigadas lo implementa (`ServicioAsignacion ..|> ConsultaZonasAsignadas`). Un recorrido del grafo de dependencias confirma que no quedan ciclos; el orden, de abajo hacia arriba, es compartido → reportes → priorización → mapa y brigadas.

Decisión registrada en `docs/architecture/adr/004-puerto-priorizacion-brigadas.md`:

```
# ADR-004: Puerto ConsultaZonasAsignadas para eliminar el ciclo entre
#          priorización y brigadas
- Estado: Aceptado
- Fecha: 2026-10-11
- Decisores: Oliva Valdivia David Alexander
## Decisión
Usaremos el puerto ConsultaZonasAsignadas, definido en el módulo de priorización e
implementado por ServicioAsignacion en el módulo de brigadas. El módulo de
priorización no importará ninguna clase del módulo de brigadas.
```

Commit: 1968714 "E5: Paquetes" (el archivo se renombró a su nombre actual en 23ae80e). La fecha del ADR es 2026-10-11 en todas las versiones del historial.

E6: Round-trip con IA

1. Adapte el Prompt IA 2 y genere el esqueleto del código de su diagrama de clases en src/<modulo>/, en Python o Java.

```
Genera el esqueleto en Python 3.10 (dataclasses y type hints) de nuestro diagrama
de clases. Respeta exactamente los nombres de clases, atributos y operaciones (en
snake_case), las enumeraciones, la interfaz como clase abstracta (ABC) y las
multiplicidades. Implementa solo la lógica mínima de las operaciones del Pedido;
los adaptadores deben lanzar NotImplementedError.
```

Resultado: `src/modulo/reportes.py`. "Respetar las multiplicidades" se refiere al **código**: se expresan con tipos (`list[Reporte]` para `0..*`, `datetime | None` para `0..1`). Como el prompt volvía a mencionar "Pedido", la

lógica mínima se aplicó a `Reporte`.

2. **Python: ejecute pyreverse -o puml -p <modulo> src/<modulo>. Java: use IntelliJ IDEA (Diagrams > Show Diagram) o el plugin PlantUML Parser y exporte el diagrama.**

pyreverse generó `src/modulo/classes_en.puml` (Figura 6). Todas sus relaciones son flechas simples sin números: **pyreverse no reconstruye las multiplicidades**.

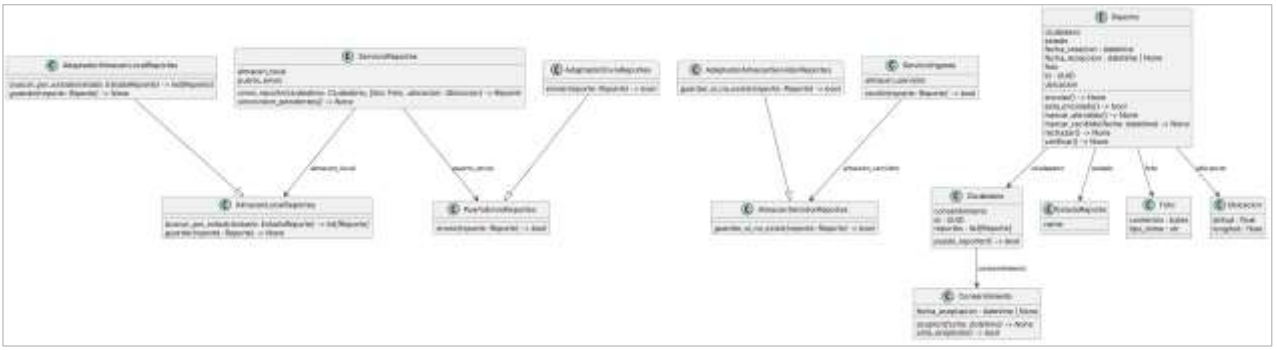


Figura 6. Diagrama generado por pyreverse desde el código (src/modulo/classes_en.puml).

3. **Compare ambos diagramas y registre en round-trip.md 3 o más diferencias, con su causa y la acción tomada (corregir el diagrama, corregir el código o documentar la limitación).**

Tabla 11. Diferencias registradas en docs/design/round-trip.md (9 en total).

#	Diferencia	Causa	Acción tomada
1	El código tiene tres adaptadores que el diseño no tenía.	El esqueleto exigía adaptadores con `NotImplementedError`.	Corregir el diagrama: se agregaron a `clases.puml` con realización (`b84689c`).
2	Los puertos aparecen como `class`, sin `<<puerto>>`.	Python no tiene interfaces; un puerto es una clase ABC.	Documentar la limitación.
3	`EstadoReporte` aparece sin sus seis valores.	pyreverse no lista los valores de un `Enum`.	Documentar; se comprobó a mano que coinciden.
4	Se pierden las multiplicidades y la composición.	Python no las declara; pyreverse solo ve atributos con tipo.	Documentar: el diseño es la única fuente de las multiplicidades.
5	Faltan las dependencias de uso (`..>`).	Son parámetros o retornos; pyreverse solo dibuja herencia y atributos.	Documentar la limitación.
6	Operaciones concretas marcadas `{abstract}`.	pyreverse marca así los métodos que lanzan `NotImplementedError`.	Ninguna: desaparece al implementarlas.
7	Atributos con nombres de rol que el diseño no tenía.	El diseño dibujaba las asociaciones sin nombre de rol.	Corregir el diagrama: se agregaron los roles (`b84689c`).
8	Nombres y tipos en `snake_case` y tipos de Python.	Convención del lenguaje.	Ninguna: traducción uno a uno.
9	No se muestra la visibilidad.	Python no tiene modificadores de acceso.	Documentar la limitación.

La diferencia 4 coincide con la respuesta a la pregunta 5 del cuestionario. Commit: `23ae80e` "E6: Round-Trip con IA"; las acciones 1 y 7 se aplicaron en `b84689c`.

E7: Revisión de consistencia y bitácora de IA

1. **Aplique el Prompt IA 3 a sus cinco diagramas y copie los hallazgos en consistencia.md.**

```
Actúa como revisor de diseño. Te paso cinco diagramas UML en PlantUML/Mermaid.
Verifica estas reglas y responde en una tabla (regla, elemento, problema,
```

```
corrección sugerida): C1 cada mensaje de secuencia es una operación de la clase
receptora; C2 cada transición de estados corresponde a una operación de la clase;
C3 multiplicidades coherentes con los criterios de aceptación; C4 paquetes sin
ciclos; C5 nombres consistentes. No reescribas los diagramas; solo reporta
hallazgos y cita la línea.
```

La IA reportó 17 hallazgos con archivo y línea, copiados en `docs/design/consistencia.md`. Tras verificarlos, el resultado por regla es:

Tabla 12. Resultado de la revisión de consistencia.

Regla	Hallazgos de la IA	Sólidos tras verificar	Estado
C1	4	3	Corregidos en b84689c (verificar_c1.py: de 6 a 0 mensajes sin operación).
C2	3	2	Pendientes: `enviar()` y `recibir()` no son operaciones de `Reporte`.
C3	2	0	Se cumple.
C4	2	0	Se cumple: sin ciclos.
C5	6	2	Corregidos en b84689c (`ENCOLADO` y `RECIBIDO` en la secuencia).

2. Verifique cada hallazgo: al menos uno debe ser un falso positivo de la IA, explicado (como en el Paso 9).

Se encontraron siete falsos positivos o discutibles; están explicados en la sección "Verificación de los hallazgos" de `consistencia.md`:

Tabla 13. Falsos positivos de la IA.

Regla	Hallazgo de la IA	Verificación
C5	Renombrar el estado "Atendido" de la asignación a "Finalizada".	Falso positivo: el nombre viene de RF-05 ("marcar su estado: en camino, atendido"); renombrarlo rompería la trazabilidad con los drivers.
C3	`id` de `Reporte` sin `{unique}`.	Falso positivo: la unicidad no es una multiplicidad y el criterio 3 se cubre con `guardarSiNoExiste()`.
C4	La nota de paquetes describe una cadena estricta.	Falso positivo: describe el orden de las capas y las dependencias lo respetan.
C4	Brigadas usa la entidad `Reporte` de otro módulo.	Falso positivo: la regla 3 permite usar clases públicas.
C5	`Borrador` frente a `BORRADOR`.	Falso positivo: mismo nombre con la convención de cada notación.
C5	`Ciudadano`, `Consentimiento` y `Foto` fuera de los paquetes.	Falso positivo: el alcance está declarado en el comentario del diagrama.
C2	`crearReporte()` en la transición inicial.	Discutible: es habitual etiquetarla con el evento que crea el objeto. Se mantiene.

3. Complete bitacora-ia.md con 5 o más interacciones (fecha, herramienta, prompt, propuesta, verificación, decisión).

`docs/design/bitacora-ia.md` registra 8 interacciones con todas las columnas pedidas (la herramienta en todas es Claude, de Anthropic). Resumen:

Tabla 14. Interacciones de la bitácora.

#	Fecha	Objetivo	Decisión
1	2026-10-07	Diagrama de clases de la HU-01	Se aplicaron las 8 correcciones de la Tabla 2.

2	2026-10-11	Diagrama de estados	`EstadoReporte` pasa a seis valores; tres operaciones nuevas en `Reporte`.
3	2026-10-11	Diagrama de actividades	5 cambios, incluido el paso por "Verificado"; queda pendiente la brigada que no responde.
4	2026-10-11	Paquetes y regla C4	Puerto `ConsultaZonasAsignadas` y ADR-004.
5	2026-10-11	Esqueleto en Python	Se acepta; pendiente probarlo en Python 3.10.
6	2026-10-11	Round-trip	Actualizar `clases.puml` (adaptadores y roles).
7	2026-10-11	Revisión de consistencia	7 hallazgos sólidos; 5 corregidos y 2 pendientes.
8	2026-11-13	Revisión del informe	Rama `lab05-correcciones` y etiqueta `v0.5.1-diseno`.

Commit: `6ecb88c` "E7: Bitacora de usa de IA"; la bitácora se completó en `b84689c`.

E8: README, revisión cruzada y entrega

1. Agregue al README.md la sección "Diseño UML (Lab 05)", con el diagrama de estados embebido en Mermaid y enlaces a los demás diagramas e imágenes.

La sección "Diseño UML (Lab 05)" del `README.md` contiene:

- El diagrama de estados embebido en un bloque de código `mermaid` (el mismo código de E3), que GitHub dibuja directamente.
- Una tabla con la fuente y la imagen de los diagramas de clases, secuencia, actividades, paquetes y del diagrama generado por pyreverse.
- Enlaces a `reportes.py`, `round-trip.md`, `consistencia.md`, `verificar_c1.py`, `bitacora-ia.md` y el ADR-004.

Comprobación de enlaces: la versión del commit `556f825` tenía 7 enlaces rotos (imágenes con otro nombre, `classes_en.puml` y `reportes.py` en otra carpeta, y un error de tipeo en el ADR-002). En `b84689c` se corrigieron, junto con el título del README; los 23 enlaces relativos apuntan a archivos existentes.

2. Revisión cruzada.

No se realizó: el laboratorio se desarrolló de forma individual, sin otro grupo con quien intercambiar la revisión. Como sustituto parcial, el diseño pasó por la revisión de consistencia con IA (E7) y por la validación del diagrama de actividades con un compañero (E4).

3. Cree la etiqueta de entrega: git tag v0.5-diseno y luego git push origin v0.5-diseno.

La etiqueta `v0.5-diseno` se creó sobre el commit `6ecb88c` (Figura 7), es decir, **antes** del commit de E8. Por eso se publicó la etiqueta `v0.5.1-diseno` sobre `b84689c`, que incluye E8 y las correcciones:

```
git tag -a v0.5.1-diseno -m "Lab 05: entrega corregida (incluye E8)"
git push origin lab05-correcciones v0.5.1-diseno
```

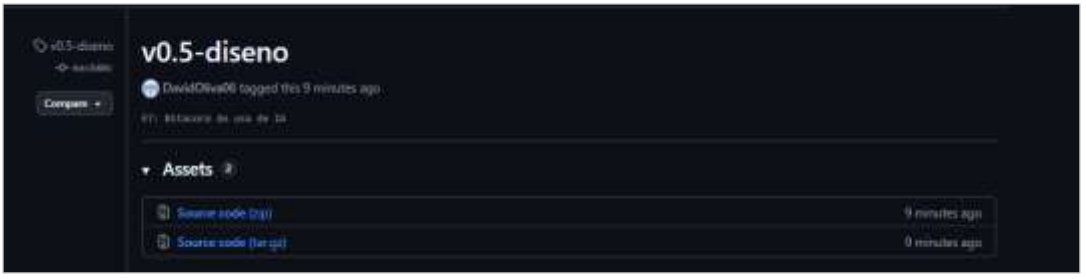


Figura 7. Etiqueta v0.5-diseno publicada en GitHub sobre el commit 6ecb88c.

II. SOLUCIÓN DEL CUESTIONARIO

¿Cuándo usaría composición y cuándo agregación? Justifique con dos relaciones de su diagrama de clases.

Uso composición cuando la parte no existe sin el todo y desaparece con él, como Reporte–Foto y Ciudadano–Consentimiento (`"1" *-- "1"`). Usaría agregación si la parte pudiera existir por su cuenta y compartirse; mi diagrama no tiene ninguna, y lo más cercano es Ciudadano–Reporte, que modelé como asociación simple (`"1" -- "0..*"`).

¿Por qué en el diagrama de clases se modela la pasarela de pagos (o el servicio externo de su caso) como interfaz y no como superclase? Relaciónelo con el ADR-001.

Porque un puerto como `PuertoEnvioReportes` solo define un contrato y no comparte código: el adaptador se puede cambiar sin tocar el dominio, y los módulos mantienen los límites claros que exige el ADR-001. Una superclase acoplaría el dominio a una implementación concreta del envío.

Explique la regla C1 y muestre un mensaje de su diagrama de secuencia que la incumplía antes de corregirlo.

C1 exige que cada mensaje del diagrama de secuencia sea una operación de la clase que lo recibe. En la versión entregada la incumplía `SVC -> REP: marcarEnviado(fecha)`: en E3 la operación pasó a llamarse `marcarRecibido(fecha)` y la secuencia no se actualizó. `verificar_c1.py` encontró 6 mensajes así; se corrigieron en el commit `b84689c` y ahora hay 0.

¿Cómo se derivan casos de prueba a partir de un diagrama de máquina de estados? Indique cuántas pruebas mínimas requiere su diagrama.

Cada transición se convierte en al menos un caso de prueba (estado de origen, operación y guarda, y estado esperado), y se agregan pruebas de transiciones no dibujadas, que deben rechazarse. Mi diagrama tiene 7 transiciones, así que requiere 7 pruebas mínimas, que se recorren en 2 caminos completos (uno termina en Atendido y otro en Rechazado).

¿Qué información del diagrama de diseño se perdió en la ingeniería inversa (E6) y por qué?

Se perdieron las multiplicidades y la composición (pyreverse dibuja todas las relaciones como flechas simples sin números), el estereotipo de interfaz, los valores de la enumeración, las dependencias de uso y la visibilidad. Ocurre porque Python no expresa esos conceptos y pyreverse solo lee clases, atributos y herencia (diferencias 2 a 5 y 9 de `round-trip.md`).

¿Qué falso positivo detectó en la revisión de consistencia con IA? ¿Qué habría pasado si lo aceptaba sin verificar?

La IA propuso renombrar el estado "Atendido" de la asignación a "Finalizada" (regla C5). Al contrastarlo con los drivers vi que el nombre viene de RF-05 ("marcar su estado: en camino, atendido"); de aceptarlo, el diagrama de actividades habría dejado de coincidir con el requisito y con lo que el brigadista ve en la app.

Según Fowler, UML puede usarse como boceto, plano o lenguaje de programación. ¿Cuál de esos

usos aplicó en este laboratorio y por qué es adecuado para un MVP de 1 mes?

Lo usé como boceto, porque los diagramas sirvieron para discutir y corregir el diseño sin detallarlo todo, y eso es adecuado para un MVP de 1 mes con un solo desarrollador, que no tiene tiempo de mantener planos completos.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] Fowler, M. (2003). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3.ª ed.). Addison-Wesley.

[2] Object Management Group. (2017). OMG Unified Modeling Language (UML), versión 2.5.1. https://www.omg.org/spec/UML/2.5.1

[3] Nygard, M. (2011). Documenting Architecture Decisions. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

[4] PlantUML. Documentación de diagramas de clases, secuencia y actividades. https://plantuml.com

[5] Mermaid. State diagrams. https://mermaid.js.org/syntax/stateDiagram.html

[6] Pylint. Pyreverse. https://pylint.readthedocs.io/en/stable/additional_tools/pyreverse/

[7] Oliva Valdivia, D. (2026). Repositorio cs-2026b-lab04-grupo06, etiqueta v0.5.1-diseno. https://github.com/DavidOliva06/cs-2026b-lab04-grupo06